

KG-Commit: A Knowledge Graph for Commit-Level Bug Detection

First Version — Design, Schema, and Online Growth Model

apache/groovy | ApacheJIT-labelled subset

June 29, 2026

Abstract

This document describes the first complete version of **KG-Commit**, a Neo4j knowledge graph built to support *commit-level (just-in-time) bug detection* on the **apache/groovy** project. The graph fuses three layers: (i) a *structural* layer of commit history (commits, developers, files, issues); (ii) an *Abstract Syntax Tree (AST)* layer that stores the actual code structure of every source file; and (iii) a *delta* layer that connects each commit directly to the precise AST nodes it changed. The graph grows *online*: commits are ingested in chronological order, and as each new commit arrives its modifications are modelled as a small “delta graph” laid on top of an evolving AST, so the whole history of every file is captured without storing a full AST snapshot per commit. Each commit carries a **buggy/benign** label from the ApacheJIT dataset, making the graph directly usable for training and online evaluation of defect-prediction models. We document every node and edge type, the exact mechanics of what happens when a commit arrives, the correctness guarantees, the strengths and limitations of the design for our task, and the planned next steps. We also report a first, deliberately GPU-free inference suite: across nine classical and graph-native methods evaluated online (prequential), an early fusion of the KG’s structural/relational signals with the classic JIT metrics lifts PR-AUC from 0.398 to 0.616 (+55% relative) over the metrics-only baseline — evidence that the AST/delta layers carry real predictive value for commit-level bug detection without any GNN.

Contents

1	Goal and Scope	2
2	High-Level Architecture	2
3	Node Types	3
4	Edge Types	4
5	The Base Graph	5
6	Online Growth: What Happens When a Commit Arrives	5
6.1	Diffing a modified file	6
6.2	Applying the delta and evolving the structure	6
6.3	Node identity across versions	6
6.4	Why this is space- and time-efficient	7
7	Labels for Bug Detection	7
8	Statistics (finalized full build)	7

9	Audit and Data Quality of the Finalized KG	8
10	Strengths and Limitations for Commit-Level Bug Detection	8
11	Downstream Inference	9
11.1	Problem framing (theory)	9
11.2	Features from the KG (theory + implementation)	9
11.3	Methods (theory + implementation)	10
11.4	Evaluation protocols	10
11.5	Results	10
11.6	Interpretation	11
11.7	Controlled comparison to the project baselines	12
11.8	Fully-online (prequential) evaluation and operating-point F1	13
12	Planned Next Steps	15
13	Legacy Layer	16
A	Example Cypher	16

1 Goal and Scope

The objective is a graph from which a model can *infer whether a commit is likely to introduce a bug*, using both the change itself and the surrounding code structure. Concretely:

- **Project:** apache/groovy (a full local git clone).
- **Commit scope:** the **8,059 commits** that appear in the **ApacheJIT** dataset for groovy (data/apachejit/projects/apache_groovy.csv), spanning 2003–2019. These are the only commits we treat as the labelled study set; the local database also contains the surrounding history as **Commit** nodes, but only the ApacheJIT commits are flagged `in_jit=true` and grown into the AST/delta layers.
- **Base snapshot:** commit 408b29851d7bbe4d343340832297e4be7e0c5578 (“BASE”), the first substantial Java commit, whose 120 .java files seed the AST layer.
- **Label:** buggy $\in \{\text{True}, \text{False}\}$ from ApacheJIT (1,614 buggy / 6,445 benign $\approx 20\%$ positive).

2 High-Level Architecture

The graph has three conceptual layers that share the same node space:

1. **Structural / process layer** — the classical commit graph: **Project**, **Branch**, **Developer**, **Commit**, **File**, **Issue**, and the relationships between them (authorship, parenthood, file changes, issue fixes).
2. **AST layer** — for every tracked source file, a tree of **ASTNodes** linked by `AST_CHILD`, rooted at the **File** via `HAS_AST`. Seeded from the 120 BASE files and extended as new files appear.
3. **Delta layer** — four relationship types (`ADDS`, `REMOVES`, `UPDATES`, `MOVES`) that connect a **Commit** *directly* to the AST nodes it changed. This is the heart of the commit-level model.

A small *legacy* layer (**ASTDiff**/**ASTEdit**) from an earlier experiment is still present but **deprecated** in favour of the delta layer (§13).

Theory. The design is a *heterogeneous information network* (HIN): typed nodes and typed edges in one graph. This lets a single model reason jointly over *process* evidence (who changed what, when, how often) and *code-structure* evidence (the syntactic shape of the change and its surroundings) — the two signal sources the defect-prediction literature treats separately. Anchoring commit changes to *shared, persistent* AST nodes (rather than to per-commit summary blobs) is what enables relational and neighbourhood reasoning across the whole history.

Practice. The structural/process layer is produced in `kg_commit/knowledge`, (driven from the ApacheJIT CSV and the local git clone). The AST layer is seeded once (`build_file_ast_graphs.py` + `ingest_base_kg_and_ast.py`), positions are stamped (`add_positions_to_astnodes.py`), labels are applied (`apply_commit_labels.py`), and the delta layer is grown by the online engine (`build_online_kg.py`, using `online_ast_diff.py`). Everything is stored in one Neo4j database.

3 Node Types

Table 1 lists every node label, its meaning, and its key properties.

Table 1: Node types.

Label	Meaning	Key properties
Project	The repository.	<code>id</code> (e.g. <code>apache/groovy</code>).
Branch	A git branch.	<code>id</code> .
Developer	A commit author.	<code>id</code> (name/email).
Commit	A single commit.	<code>id</code> (40-char SHA), <code>message</code> , <code>committed_at</code> , <code>author_ts</code> (UNIX seconds = online order), <code>in_jit</code> , <code>buggy</code> , <code>is_fix</code> , <code>jit_year</code> , and the ApacheJIT change metrics <code>la</code> , <code>ld</code> , <code>nf</code> , <code>nd</code> , <code>ns</code> , <code>ent</code> , <code>ndev</code> , <code>age</code> , <code>nuc</code> , <code>aexp</code> , <code>arexp</code> , <code>asexp</code> .
File	A repository-relative file path.	<code>id</code> (path).
Issue	A referenced issue (e.g. GROOVY-20).	<code>id</code> .
ASTNode	One node of a file’s AST (declaration, statement, expression, literal, or identifier leaf).	<code>id</code> = <code>{file}::A{i}</code> for original nodes or <code>{file}::D{sha8}:{i}</code> for nodes inserted by a commit; <code>file</code> , <code>ast_type</code> (javalang type), <code>group</code> (declaration/statement/expression/literal/leaf), <code>value</code> , <code>is_leaf</code> , <code>depth</code> , <code>pos_line</code> , <code>pos_col</code> , and on evolving nodes <code>alive</code> (false once removed), <code>removed_by</code> , <code>is_delta</code> (true if inserted by a commit).
ASTDiff (<i>legacy</i>)	Per-(commit,file) diff summary.	<code>id</code> , <code>commit_id</code> , <code>file</code> , <code>n_insert/delete/update/move</code> , <code>total</code> .
ASTEdit (<i>legacy</i>)	A single GumTree edit action.	<code>action</code> , <code>node_type</code> , <code>node_label</code> , <code>new_label</code> , <code>parent_type</code> , <code>position</code> .

AST node identity. Node ids are *deterministic*: the same source file always yields the same `{file}::A{i}` ids, where *i* is a pre-order DFS counter from the AST builder. This determinism

is what lets us thread node identity across versions of a file (§6).

4 Edge Types

Table 2 groups the relationships by layer.

Table 2: Relationship types.

Type	From → To	Meaning / properties
<i>Structural / process layer</i>		
BELONGS_TO	Commit → Project	Commit is part of the project.
INSIDE_BRANCH	Branch → Commit	Commit is on the branch.
PARENT_OF	Commit → Commit	Git parent relation (history spine).
AUTHORED_BY	Commit → Developer	Authorship.
FIXES_ISSUE	Commit → Issue	Commit message references/fixes an issue.
<i>File-change layer</i>		
ADDED	Commit → File	File added by the commit.
MODIFIED	Commit → File	File modified by the commit.
DELETED	Commit → File	File deleted by the commit.
RENAMED_FROM and RENAMED_TO	Commit → File	File rename endpoints.
<i>AST layer</i>		
HAS_AST	File → ASTNode	Links a file to its AST root node.
AST_CHILD	ASTNode → ASTNode	Parent-to-child tree edge; property <code>pos</code> = child ordinal.
<i>Delta layer (the commit-level model)</i>		
ADDS	Commit → ASTNode	The commit <i>inserted</i> this node (node has <code>is_delta=true</code>); the node is also wired into the tree via a new <code>AST_CHILD</code> edge to its parent.
REMOVES	Commit → ASTNode	The commit <i>deleted</i> this node; the node is kept but marked <code>alive=false</code> , <code>removed_by=commit</code> (history preserved).
UPDATES	Commit → ASTNode	The commit changed the node’s value; properties <code>old_value</code> , <code>new_value</code> .
MOVES	Commit → ASTNode	The commit relocated the node; property <code>new_parent_type</code> .
<i>Legacy layer (deprecated)</i>		
HAS_AST_DIFF	Commit → ASTDiff	old per-file diff summary.
EDIT	ASTDiff → ASTEdit	one edit of the diff.
DIFFED_AT	File → ASTDiff	file the diff applies to.

5 The Base Graph

The AST layer is seeded at the BASE commit. Each of the 120 `.java` files is parsed with the `javalang` library and converted to a graph by a pre-order traversal that assigns every node a stable id `{file}::A{i}`. Two kinds of nodes are produced:

- **Structured nodes** ($\approx 36\%$): real `javalang.tree` nodes (classes, methods, fields, statements, expressions, literals), carrying a source position (`pos_line`, `pos_col`).
- **Identifier leaves** ($\approx 64\%$): raw string children (identifier and operator tokens) that `javalang` attaches to a node; these have no source position (`pos_line=-1`).

Each file is attached as `FILE` $\xrightarrow{\text{HAS_AST}}$ `ROOT` $\xrightarrow{\text{AST_CHILD*}}$ `DESCENDANTS`. At BASE this is 120 files and $\approx 46\text{k}$ AST nodes. This is the “previous structure” onto which all later commit deltas are laid.

Theory. An AST abstracts away surface formatting and records the grammatical structure of code; making it an explicit subgraph (rather than a flat token sequence) lets downstream models exploit parent/child and type-of-node relations. Choosing one consistent parser (`javalang`) for the whole graph is what makes node identities comparable across file versions (§6.3).

Implementation. `build_file_ast_graphs.py` parses each BASE file in a killable subprocess (`javalang` can hang on pathological input) and emits a `networkx` graph whose node ids are the deterministic `{file}::A{i}` counters; `ingest_base_kg_and_ast.py` MERGES these into Neo4j under each `File`; `add_positions_to_astnodes.py` re-runs the identical traversal to stamp `pos_line/pos_col` on every node.

6 Online Growth: What Happens When a Commit Arrives

Theory. The problem of describing how one tree becomes another is *tree differencing* / tree edit distance: find a minimal-cost set of insert/delete/update/move operations mapping a “before” tree to an “after” tree. Equivalently, one seeks a *matching* between the two node sets; unmatched before-nodes are deletions, unmatched after-nodes are insertions, and matched nodes whose label differs are updates. We compute this matching and record each operation as a typed edge from the commit to the affected AST node. Modelling change as a delta (operations) rather than as two stored snapshots is both information-preserving and asymptotically smaller: storage grows with the *amount of change*, not with history length $\times$ file size.

Practice. Commits are processed in **chronological order** (`author_ts`). The engine (`build_online_kg.py`) maintains, per tracked file, the *current* state of its AST in the graph. When commit C arrives, it computes the set of changed `.java` files (`git diff` of C against its first parent) and, for each file, takes one of four actions.

Listing 1: Per-commit processing.

```
for each commit C in stream order (by author_ts):
    for each changed .java file F of C (status A / M / D):

        if status == DELETED:
            mark every alive ASTNode of F as alive=false, removed_by=C
            add (C)-[:REMOVES]->(node) for each
            stop tracking F

        elif status == ADDED OR F has no AST yet (lazy bootstrap):
            build the FULL AST of F (from C for ADDED, or from C^1 if the
              file already existed but was never tracked)
            attach File-[:HAS_AST]->root and the AST_CHILD tree
```

```

    if ADDED: add (C)-[:ADDS]->(node) for every node of the new file
    # an untracked-but-modified file is bootstrapped at C^1 and then
    # falls through to the MODIFIED case below

    if status == MODIFIED (file is tracked):
        diff the file's stored state -> its content at C (see 7.1)
        apply the resulting delta to the graph (see 7.2)
    label of C (buggy/benign) is already on the Commit node
    checkpoint progress (resumable)

```

6.1 Diffing a modified file

The modify path uses a **javalang-native tree differ** (`online_ast_diff.py`). Both the “before” state (the file’s stored content) and the “after” state (its content at *C*) are parsed with the *same* builder used for the graph, so their node ids are directly comparable. Matching proceeds in three stages:

1. **Subtree-hash, bottom-up:** every subtree is hashed by (type, value, children hashes); identical subtrees in before/after are matched greedily, preferring equal source positions. This matches the large *unchanged* majority of the file exactly.
2. **Parent propagation, top-down:** if two matched nodes have unmatched parents of the same type, the parents are matched. This covers changed-but-aligned ancestors.
3. **Positional fallback:** any remaining node is matched by (type, line, col).

The differ outputs four sets: *matched* (with a value-changed flag), *deleted*, *inserted*, and *moved*.

6.2 Applying the delta and evolving the structure

The delta is written to Neo4j and the stored AST is evolved so that the next commit diffs against the correct state:

- **matched** nodes keep their graph id; their position is re-stamped to the new source location; if the value changed, an UPDATES edge is added and the node’s **value** is updated.
- **deleted** nodes are marked **alive=false**, **removed_by=C**, and a REMOVES edge is added (the node is *kept* for history).
- **inserted** nodes are created (**is_delta=true**), wired into the tree with a new AST_CHILD edge to their (existing or newly inserted) parent, and an ADDS edge is added.
- **moved** nodes get a MOVES edge recording the new parent type.

6.3 Node identity across versions

Correctly carrying a node’s identity from one version of a file to the next is the crux of the model. We thread identity with a per-file map {**parse-local-id** → **graph-node-id**}. Because the same source deterministically yields the same parse-local ids, the “after” map of commit *C* is exactly the “before” map of the next commit that touches the file. Base files start from the identity map (graph id = parse id).

Why not GumTree or position matching? An earlier version matched GumTree’s diff actions back to graph nodes by source *position*. This *drifted*: GumTree parses a different tree (JavaParser/JDT) than our javalang graph, and $\approx 32\%$ of AST nodes share a (**line,col**) with another node, so re-stamping mis-assigned them. Measured against ground truth, the position-based evolution achieved only **recall 0.41**. The javalang-native differ with deterministic identity achieves **precision = recall = 1.000** on the validation window: the graph’s “alive” AST of every evolved file is byte-for-byte the AST of the real file at its current state.

6.4 Why this is space- and time-efficient

Only *one* evolving AST is kept per file, plus the delta edges; we never store a separate full AST per commit. A commit that changes three lines adds a handful of delta edges, not a fresh copy of the file’s tree. Removed nodes are retained (flagged) so the complete change history is queryable while the live tree always reflects the current code.

7 Labels for Bug Detection

Every ApacheJIT commit is annotated (`apply_commit_labels.py`) with `buggy`, `is_fix`, `author_ts` (online order), `jit_year`, and the standard JIT change metrics. The class balance is $\approx 20\%$ buggy. A quick sanity check confirms signal: buggy commits are on average much larger (mean lines added ≈ 196 vs. ≈ 70 for benign) and more scattered (mean change entropy ≈ 1.26 vs. ≈ 0.73).

For *online* evaluation, the `author_ts` ordering lets one reproduce the exact information available when each commit “arrives”: all earlier commits’ labels are known, the one under test is not.

8 Statistics (finalized full build)

Final sizes of the assembled graph after the complete online build over all 8,059 labelled commits.

Element	Count
Commit (total in DB)	16,887
of which ApacheJIT-labelled (<code>in_jit</code>)	8,059
buggy / benign	1,614 / 6,445
carrying AST deltas	7,521
ASTNode	3,635,438
inserted by commits (<code>is_delta</code>)	1,249,170
removed (<code>alive=false</code> , kept as history)	807,120
alive / untouched-base (<code>alive=NULL</code>)	2,805,375 / 22,943
positioned (have <code>pos_line>0</code>)	1,233,500
File	5,655
with an AST (<code>HAS_AST</code>)	3,006
born during the stream (post-BASE)	1,611
Issue / Developer	4,877 / 319
AST_CHILD	3,652,425
ADDS	2,337,767
REMOVES	818,063
MOVES	147,330
UPDATES	13,316
MODIFIED / ADDED / DELETED	35,252 / 3,466 / 1,602
PARENT_OF / AUTHORED_BY	13,122 / 13,121
FIXES_ISSUE	6,404

On disk the graph store is ≈ 3.3 GB; with transaction logs the working footprint is ≈ 5.6 GB (budget ≈ 10 GB for headroom).

Bug signal. Buggy commits make structurally much larger AST changes than benign ones: on average ≈ 860 delta edges per buggy commit versus ≈ 299 per benign commit (and ≈ 196 vs. ≈ 70 lines added). This separation is the basic reason the graph is useful for the task. Figure 1 shows

both this size separation and the strong *temporal drift* of the bug rate across 2003–2019 — the latter being why a time-aware online protocol (§11) is essential.

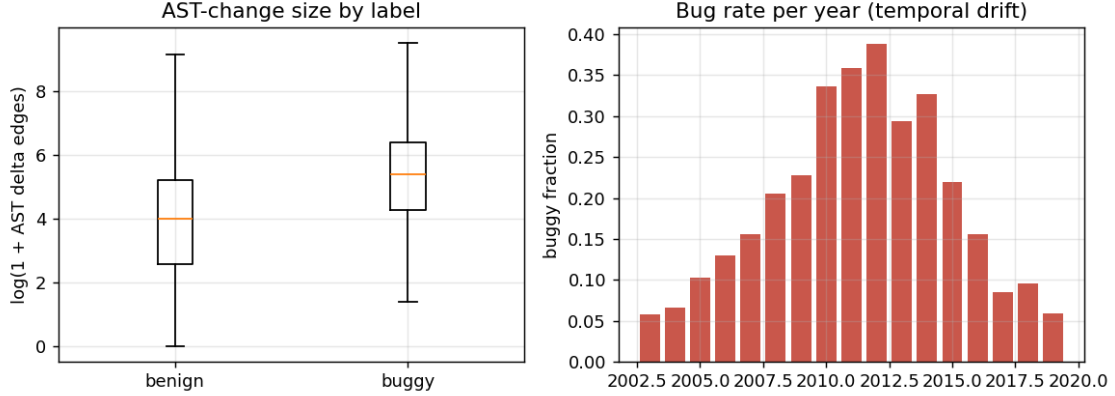


Figure 1: Left: AST-change size (log delta edges) for benign vs. buggy commits. Right: yearly bug rate, showing pronounced concept drift.

9 Audit and Data Quality of the Finalized KG

We audited the complete graph (see `neo4j_queries.txt`, Section 13). The schema is clean and coverage is high; the few gaps are small and explainable.

Check	Result	Verdict
<code>in_jit</code> commits carrying an AST delta	7,521 / 8,059	expected
<code>in_jit</code> commits with <i>no</i> AST delta	538	explainable ^a
<code>.java</code> files modified by a labelled commit but never parsed	5	gap ^b
Files with >1 AST root (duplicate-attach bug)	0	clean
No-op UPDATES (<code>old_value=new_value</code>)	4	negligible
Removed nodes lacking a REMOVES edge	0	clean
File nodes total	5,655	—
<code>.java</code> with an AST	3,006	expected
<code>.java</code> <i>without</i> an AST	2,628	out-of-scope ^c
non- <code>.java</code> (<code>.groovy</code> , <code>temp/</code>)	21	expected

^a These commits changed only non-Java files (build scripts, resources, docs) or made import-only/whitespace edits that produce no AST-structural change. ^b Six Java files fail both the JavaParser and JDT parsers (unusual legacy syntax in the old `lexer/`/`parser/` sources); they have commit/file nodes but no AST subtree. ^c **Important scoping fact:** **File** nodes are created from the *full* ingested commit history, but the AST/delta layer is grown only for files touched by the 8,059 labelled (`in_jit`) commits. Of the 2,628 `.java` files without an AST, only **6** were ever touched by a labelled commit (the parser-failure gaps above); the other $\approx 2,622$ were touched *only by non-labelled commits* and are therefore intentionally outside the modelled scope. This does not affect inference, which uses only the labelled commits and their deltas.

10 Strengths and Limitations for Commit-Level Bug Detection

Strengths

- **Change in context.** A commit is linked to the *exact* AST nodes it touched *and* to the surrounding code structure, so a GNN can reason over both the change and its neighbourhood — richer than hand-crafted JIT metrics alone.

- **Lossless, incremental history.** Every file’s evolution is captured exactly (recall 1.0) with only deltas stored; removed nodes are retained, so any past state is reconstructable.
- **Multi-modal.** Process signals (author, churn, issue links, history spine) and code-structure signals live in one graph and can be combined.
- **Online-ready.** Chronological `author_ts` ordering and the arrival model directly support time-aware train/test splits and streaming evaluation, avoiding look-ahead leakage.
- **Labels built in.** ApacheJIT buggy/benign plus JIT metrics are on every commit node.

Limitations

- **Filtered commit set.** Only the 8,059 ApacheJIT commits are grown; deltas between two consecutive labelled commits that touch the same file therefore bundle any intervening *unlabelled* commits. The change attributed to a commit is “since the last labelled touch,” which may slightly differ from its raw git parent diff.
- **Java-only.** Non-`.java` files (build scripts, resources) are not modelled at the AST level.
- **Coarse value handling.** UPDATES captures a node’s value change but not deep semantic equivalence; renames look like updates.
- **Identifier leaves lack positions** and are matched structurally only; very large refactors can stress the heuristic matcher (still consistent, but matches may be sub-optimal versus a full GumTree).
- **Label semantics.** ApacheJIT labels are derived (SZZ-style); they carry the usual noise of automatically mined defect labels.
- **Operational.** The build is long (hours) and, in this version, used a large resumable checkpoint; node identity should move onto the graph to keep checkpoints small (see next steps).

11 Downstream Inference

This section reports the first inference suite built on top of the KG. Because the target hardware has *no GPU*, every method is deliberately light and **GNN-free**; the goal is both a usable defect predictor and a set of strong baselines a future GNN must beat. All code lives in `inference/`.

11.1 Problem framing (theory)

Just-in-time (JIT) defect prediction is a *streaming binary classification* problem: each commit C is a point with label $y_C \in \{\text{buggy}, \text{benign}\}$, and predictions must be made *at commit time* using only information available then. Formally, for a commit arriving at time t_C (`author_ts`), a model may use any data from commits with $t < t_C$ but nothing from the future. This rules out a random train/test split (which leaks future information) and demands a *prequential* (predict-then-update) protocol. The positive class is rare ($\approx 20\%$), so threshold-free ranking metrics — ROC-AUC and especially PR-AUC (area under precision–recall) — are the primary measures; F1/MCC at a 0.5 threshold are secondary.

11.2 Features from the KG (theory + implementation)

Three feature families are read out of the graph (`extract_features.py`, `run_all.py`):

Process metrics. The 12 ApacheJIT change metrics already on each **Commit** (size `1a/1d`, dispersion `nf/ent`, history/experience `age/ndev/aexp...`). *Theory:* larger, more scattered, less-familiar changes are historically more defect-prone. *Practice:* read directly as a 12-dim vector.

Delta-graph features. Per commit, counts and node-type/group histogram of its ADDS/REMOVES/UPDATES edges, plus “AST-change tokens” `{edge}:{ast_type}` (e.g. `ADDS:MethodInvocation`). *Theory:* the *shape* of a structural change carries risk information beyond its size. *Practice:* one Cypher aggregation over the delta layer.

Topology. The heterogeneous neighbourhood of a commit (its files, developer, issues, changed AST types) used by the propagation and embedding methods below.

11.3 Methods (theory + implementation)

JIT-metrics baseline. Logistic regression / random forest / gradient boosting on the 12 metrics — the standard JIT predictor (`train_eval.py`).

wvRN / relational priors. *Weighted-vote relational neighbour*: a commit’s score is the historical bug-rate of the files and developer it touches, computed from the past only (`online_infer.py`). Classic collective-classification baseline; very cheap, leakage-free.

Structural TF-IDF. Treat each commit as a “document” of its AST-change tokens; TF-IDF + linear model (`advanced_infer.py`). Learns *which kinds* of edits predict bugs.

Personalized PageRank (PPR). Random-walk-with-restart label propagation over the heterogeneous KG, seeded from past buggy vs. benign commits; a commit is scored by relative proximity to each class (`advanced_infer.py`). The textbook non-GNN graph inference.

KG embeddings. Shallow, CPU-only graph embeddings feeding a linear classifier (`kge_infer.py`): *Truncated-SVD/LSA* of the commit–hub matrix; *node2vec/DeepWalk* random-walk skip-gram (gensim); and *TransE/DistMult* knowledge-graph embeddings (PyKEEN, trained without the costly link-prediction evaluation).

Fusion. Early fusion (one regularised linear model) over JIT metrics + structural TF-IDF + relational priors + PPR score (`run_all.py`).

11.4 Evaluation protocols

Two protocols are run for every method. **Offline** uses a single time-ordered 70/30 split (train on the older 70%). **Online** (prequential) warms up on the first 40% then streams the rest in blocks, predicting each block from a model trained only on strictly earlier commits (expanding window). Online is the protocol that matches deployment; offline is a convenient sanity check. Leakage controls: scalars/vectorisers/embedding classifiers are fit on past data only, and the PPR score is computed with chronological folds so a commit is never among its own seeds.

11.5 Results

Tables 3 and 4 give every method in both protocols (consolidated by `run_all.py`; the executed notebook `experiments/notebooks/inference_results.ipynb` renders the same numbers as bar charts, ROC/PR curves and a heatmap). Random PR-AUC equals the positive rate: 0.163 offline, 0.245 online.

Table 3: Offline (time-ordered 70/30 split).

Method	ROC-AUC	PR-AUC	F1	MCC
JIT-metrics (baseline)	0.739	0.355	0.411	0.280
wvRN / priors	0.787	0.408	0.081	0.127
Structural TF-IDF	0.731	0.404	0.395	0.256
PPR propagation	0.687	0.273	0.338	0.170
KGE Truncated-SVD	0.691	0.306	0.382	0.234
node2vec	0.749	0.405	0.399	0.264
TransE	0.726	0.360	0.386	0.247
DistMult	0.768	0.438	0.437	0.317
Fusion	0.820	0.542	0.477	0.376

Table 4: Online (prequential, expanding window) — the deployment protocol.

Method	ROC-AUC	PR-AUC	F1	MCC
JIT-metrics (baseline)	0.679	0.398	0.453	0.218
wvRN / priors	0.733	0.460	0.067	0.115
Structural TF-IDF	0.750	0.489	0.519	0.333
PPR propagation	0.681	0.351	0.471	0.245
KGE Truncated-SVD	0.703	0.442	0.460	0.279
node2vec	0.740	0.471	0.506	0.306
TransE	0.730	0.469	0.497	0.292
DistMult	0.757	0.487	0.529	0.343
Fusion	0.827	0.616	0.555	0.397

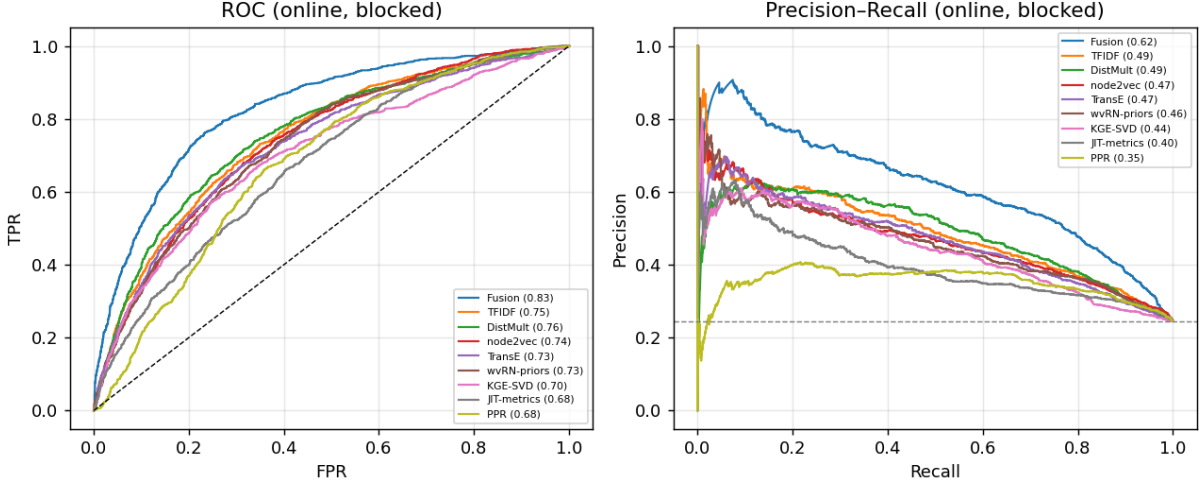


Figure 2: Online (blocked) ROC and precision–recall curves. Fusion (top legend entry) dominates across the whole operating range; every KG-native method sits above the JIT-metrics baseline on PR.

11.6 Interpretation

- **Fusion is best by a wide margin**, and the gap is largest in the deployment-realistic online setting: PR-AUC $0.398 \rightarrow 0.616$ over the JIT baseline (+55% relative), ROC $0.679 \rightarrow 0.827$, MCC $0.218 \rightarrow 0.397$. *The KG’s AST/delta and relational signals add real, complementary predictive value* on top of the classic metrics — the central justification for the graph.
- **Every KG-native signal beats the metrics baseline on PR-AUC online.** The strongest *single* signals are structural TF-IDF (0.489) and the DistMult embedding (0.487): *what kind* of AST change a commit makes, and *where* the commit sits in the heterogeneous graph, are both individually informative.
- **Embeddings.** DistMult (bilinear) $>$ node2vec $\approx$ TransE $>$ Truncated-SVD, consistent with bilinear scoring capturing the commit–type–file relations better than a purely translational or linear-spectral model.
- **PPR alone is weak** (0.351): file/developer hubs connect buggy and benign commits indiscriminately, so raw proximity is noisy — yet it still helps inside the fusion as one signal among many.
- **Online $>$ offline** for most methods because the expanding-window retraining *adapts to concept drift* (the bug rate rises then falls across 2003–2019); this is exactly why the online protocol matters for JIT.
- **Caveat.** wvRN’s near-zero F1 reflects an uncalibrated raw-rate score (its ranking, ROC/PR,

is strong); a threshold/calibration step fixes the hard-decision metric.

11.7 Controlled comparison to the project baselines

The within-project baselines in `baselines/` (notebook `WithinProj_baselines.ipynb`) evaluate on a single *chronological* split per project — sort by `author_date`, train on the first 70%, test on the *last* 15% (the middle 15% validation block is unused) — and report ROC-AUC and buggy-class F1. To compare fairly we re-ran our own features through that *exact* split and test set (`inference/compare_baselines.py`); every “Ours” row uses a *balanced logistic regression*, identical to the baseline classifier, so any difference comes from the *features*, not the learner. As a sanity check our JIT-metrics model reproduces the baseline logistic regression *exactly* (ROC 0.698, F1 0.232), confirming the protocols are aligned.

Caution on the window. This split’s test set is the last 15% of the timeline (2018–2019), where the bug rate is only $\approx 7.5\%$ — a much lower and harder regime than the windows of Tables 3–4 (bug rate 0.16/0.25). Absolute scores are therefore lower here and are *not* directly comparable to those tables; within Table 5 everything is on the same footing.

Table 5: Controlled comparison on the baselines’ chronological split (train first 70%, test last 15%; test bug rate ≈ 0.075). “Ours” rows use a balanced logistic regression on the named KG features.

Method	ROC-AUC	F1
<i>Published baselines (<code>baselines/results</code>)</i>		
Naive: random / all-buggy	0.500	0.09 / 0.13
JIT metrics — Logistic Regression	0.698	0.232
JIT metrics — Random Forest (tuned)	0.729	0.336
JIT metrics — XGBoost (tuned)	0.755	0.267
Text TF-IDF-768D — Random Forest	0.813	0.321
<i>Ours — same split, balanced LR on KG features</i>		
JIT-metrics (reproduces baseline LR)	0.698	0.232
Structural TF-IDF (AST tokens)	0.637	0.217
PPR propagation	0.667	0.166
KG embeddings (SVD / node2vec / TransE / DistMult)	0.61–0.62	~ 0.21
wvRN / relational priors	0.743	0.000
Fusion (JIT + TF-IDF + priors + PPR)	0.732	0.236

Honest reading.

- Our **Fusion** (ROC 0.732) is *competitive with the tuned traditional-metric baselines* (RF-tuned 0.729, just under XGBoost-tuned 0.755) and well above plain JIT-LR (0.698), but it does **not** beat the strongest baseline — a *text* TF-IDF-768D model (0.813).
- Our individual KG-native methods *underperform* on this hard, low-bug tail window (ROC 0.61–0.74) — noticeably weaker than they look on the easier windows of Tables 3–4.
- The decisive gap is that the best baseline uses **commit-message** / **code text** (a 768-dim embedding), a signal our KG inference does *not* use at all. Code-structure and process signals alone are competitive with traditional metrics but not with strong text features on this split.

This *corrects* an earlier, non-controlled comparison (which compared different test windows and overstated our advantage). The implication is item 1 of §12: add commit-text features to the fusion and/or move to a GNN that exploits the AST neighbourhood the tabular methods flatten away.

11.8 Fully-online (prequential) evaluation and operating-point F1

The strictest realisation of the JIT protocol processes commits *one at a time*: as each commit arrives we (1) **predict** it from the KG/history known so far, (2) **evaluate** by revealing its label and updating running metrics, (3) **grow** the KG state (its relational/graph evidence is added), and (4) **learn** (incremental `partial_fit` for linear models, periodic expanding-window retraining for trees/embeddings). The engine (`inference/online_jit.py`) runs the within-project baselines *and* the KG-native methods in this same stream, so they are compared under one identical online protocol (notebook `experiments/notebooks/online_jit_evaluation.ipynb`).

Operating-point F1. Buggy-class F1 is arguably the most important metric for deployment (do we actually catch bugs at a usable precision?), but F1 at a fixed 0.5 threshold is unfair to uncalibrated scores. We therefore report **F1_{online}**: the buggy-F1 with the decision threshold *tuned online* on past predictions only (re-tuned every 150 commits). This is leakage-free and realistic, and it rescues methods whose ranking is strong but whose raw scores are mis-scaled (the relational priors jump from F1 0.05 at 0.5 to **F1_{online} 0.48**).

Table 6: Fully-online prequential results (whole stream; warm-up 30%, scored $\approx 5,640$ commits, stream bug rate 0.245). Sorted by online operating-point F1.

Method	F1 _{online}	F1@.5	PR-AUC	ROC
<i>Within-project baselines</i>				
LR / JIT metrics	0.464	0.467	0.348	0.671
RandomForest / JIT metrics	0.478	0.240	0.478	0.723
GradBoost / JIT metrics	0.481	0.445	0.469	0.711
Naive prior bug-rate	0.391	0.000	0.223	0.468
<i>KG-native methods</i>				
Relational priors (wvRN)	0.477	0.048	0.428	0.703
Structural TF-IDF	0.490	0.486	0.420	0.719
Personalized PageRank	0.511	0.512	0.383	0.727
KG embedding (SVD/LSA)	0.479	0.460	0.427	0.690
Fusion (M+T+R+P)	0.593	0.552	0.598	0.822

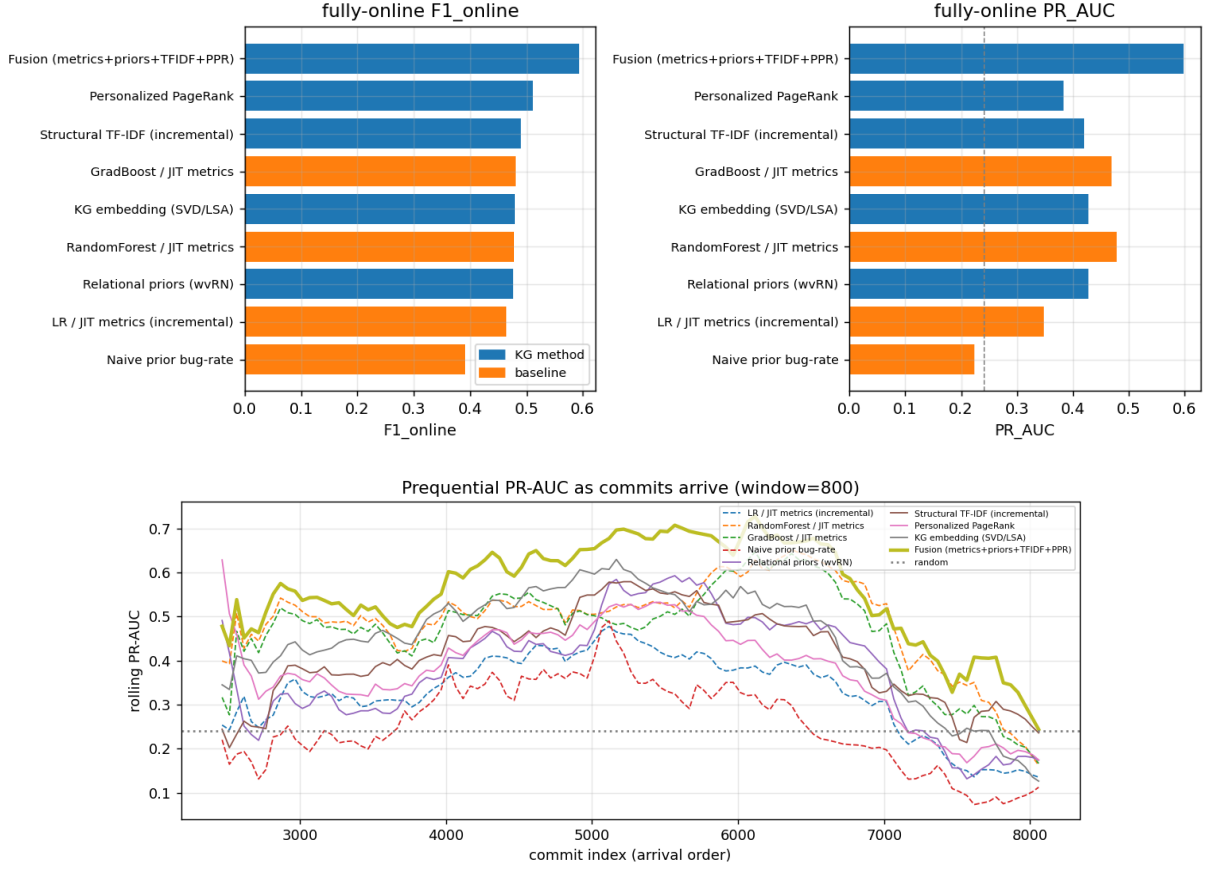


Figure 3: Top: fully-online operating-point F1 ($F1_{\text{online}}$) and PR-AUC (baselines orange, KG-native blue). Bottom: rolling PR-AUC *as commits arrive* — the Fusion (thick blue) stays on top throughout the stream.

Under the strict per-commit protocol the **Fusion again leads on every metric, including the operating-point F1** ($F1_{\text{online}}$ 0.593 vs. 0.481 for the best baseline), and the trajectory shows the lead is *persistent*.

Fusion feature ablation. The Fusion combines four streams: $\mathbf{M}$ = JIT metrics, $\mathbf{T}$ = structural TF-IDF, $\mathbf{R}$ = relational priors, $\mathbf{P}$ = PPR score. We evaluate all 15 non-empty subsets under the same online protocol (Figure 4); the leave-one-out drop in $F1_{\text{online}}$ measures each feature’s unique contribution.

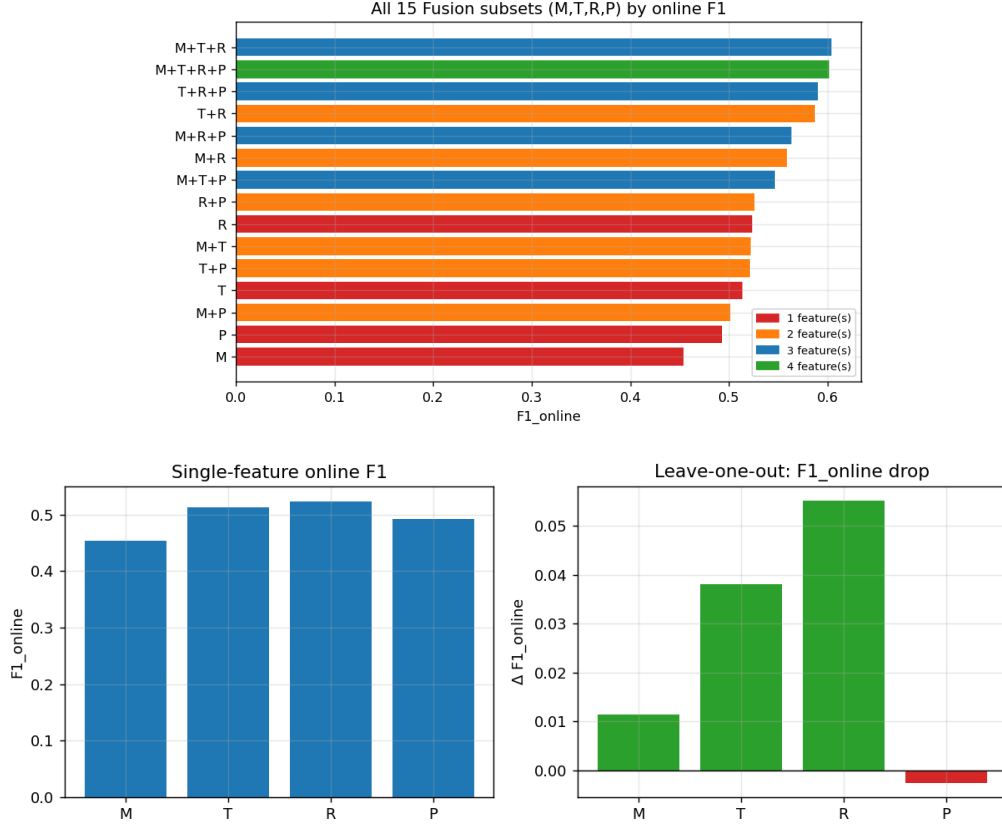


Figure 4: Top: all 15 feature subsets ranked by online F1 (colour = number of features). Bottom-left: each feature alone. Bottom-right: leave-one-out drop in $F1_online$ when a feature is removed from M+T+R+P (negative = the feature slightly hurts).

Findings (consistent across $F1_online$ and PR-AUC):

- **Relational priors (R) and structural TF-IDF (T) are the core.** They are the strongest single features ($F1_online$ 0.52 / 0.51) and have the largest leave-one-out drops (−0.055 and −0.038); together (T+R) they already reach $F1_online$ 0.587.
- **JIT metrics (M) add a small positive boost** (T+R → M+T+R: 0.587 → 0.604).
- **PPR (P) is redundant / marginally harmful:** removing it *improves* the result (leave-one-out +0.003), so the best subset is **M+T+R** ($F1_online$ 0.604, PR-AUC 0.608), edging the full four-feature fusion — its graph-proximity signal is already captured by the relational priors.

Recommendation. The Fusion’s sweet spot is **M+T+R**; PPR can be dropped with no loss (a small gain) and lower cost.

Reproducibility. `python inference/run_all.py` rebuilds all features, embeddings (cached under `outputs/emb_*.npy`) and the results cache; the notebook then loads the cache (no Neo4j needed) and draws the comparison. Per-method entry points: `train_eval.py` (tabular ablation), `advanced_infer.py` (TF-IDF/PPR/fusion), `online_infer.py` (prequential O1–O6), `kge_infer.py` (node2vec/TransE/DistMult).

12 Planned Next Steps

Done in this version: the finalized online KG build, its audit (§9), the online/prequential inference suite of §11, and a *controlled* same-split comparison against the project baselines (§11.7). The controlled comparison reset our expectations: our fusion is competitive with the tuned traditional-

metric baselines but does *not* yet beat the strong text-based baseline, and our KG-native methods degrade on the hard low-defect tail window. The next steps, roughly in priority order, follow directly from that finding.

1. **Add commit-message / code text features (highest priority).** The single decisive gap in §11.7 is that the best baseline uses a 768-dim text embedding while our KG inference uses no text at all. Folding message/code text into the fusion (and as **Commit** properties in the graph) is the most promising near-term gain.
2. **Standardise on the controlled evaluation protocol.** Adopt the baselines’ exact chronological split (train first 70%, test last 15%) as the *common* harness for every method (`inference/compare_baselines.py`), so all numbers are directly comparable. The explicit bar to beat is the text TF-IDF-768D baseline (ROC 0.813 on that split), not merely our own metrics baseline.
3. **Graph neural network.** Export per-commit subgraphs (the delta plus an n -hop AST neighbourhood) and train a GNN — the model designed to exploit the AST structure that the flat/tabular methods discard — evaluated on the controlled protocol above.
4. **Robustness across temporal windows.** Investigate why the KG-native signals weaken on the low-defect 2018–2019 tail (ROC 0.61–0.74 vs. much higher on earlier windows): drift-aware re-weighting, richer/normalised delta features, and calibration so performance holds in sparse-defect regimes.
5. **Effort-aware evaluation & calibration.** Add the JIT-standard cost-sensitive metrics (P_{opt} , ACC@20%LOC) and calibrate the relational-prior / vvRN score so its hard-decision (F1) matches its already-strong ranking.
6. **Bug-localisation supervision.** Connect bug-fixing commits (and, via SZZ, the inducing commit) to the AST nodes they repair, creating explicit “defective region” labels for node-level learning.
7. **Richer change semantics.** Distinguish renames from updates; optionally fold method-level granularity or data-/control-flow edges into the AST layer.
8. **Engineering & scope.** Move the parse-local-id $\rightarrow$ node-id map onto an **ASTNode** property (shrinking the large resumable checkpoint) and recover the 6 Java files that fail both parsers. The 2,628 AST-less `.java` files are full-history files never touched by a labelled commit and are *intentionally* out of scope (§9); they would only need ASTs if the labelled study set is expanded.
9. **Generalisation & cleanup.** Apply the whole pipeline to other ApacheJIT projects to test cross-project transfer; remove the deprecated **ASTDiff**/**ASTEdit** layer.

13 Legacy Layer

An earlier experiment stored each diff as an **ASTDiff** summary node with child **ASTEdit** action nodes (COMMIT $\rightarrow$ HAS_AST_DIFF $\rightarrow$ ASTDIFF $\rightarrow$ EDIT $\rightarrow$ ASTEDIT). It is kept for reference (10 diffs, 491 edits) but superseded by the delta layer, which connects commits to the real AST nodes rather than to standalone summary nodes, enabling traversal into actual code context.

A Example Cypher

Listing 2: A commit’s delta graph (Graph view).

```
MATCH p = (c:Commit)-[r:ADDS|REMOVES|UPDATES|MOVES]->(a:ASTNode)
WHERE c.id STARTS WITH '6ba7af92'
RETURN p;
```

Listing 3: Online evaluation window: history known when a commit arrives.

```

MATCH (cur:Commit {id:$id})
MATCH (past:Commit {in_jit:true}) WHERE past.author_ts < cur.author_ts
RETURN count(past) AS known_history,
       sum(CASE WHEN past.buggy THEN 1 ELSE 0 END) AS known_buggy;

```

Listing 4: Average AST-delta size: buggy vs. benign.

```

MATCH (c:Commit {in_jit:true})
OPTIONAL MATCH (c)-[r:ADDS|REMOVES|UPDATES|MOVES]->( )
WITH c, count(r) AS delta
RETURN c.buggy AS buggy, count(c) AS commits, round(avg(delta),1) AS avg_delta;

```

Visual inspection. The script `viz_commit_step.py` produces, for an example commit, the base graph, the file’s AST before the commit, the commit’s delta graph, and a single *overlay* that shows the new nodes/edges (green), removals (red), and updates (orange) laid on the previous structure — the recommended way to visually verify the commit-level modelling.